



Pashov Audit Group

K3 Security Review

August 19th 2026 - August 23rd 2026

Executive Summary

A time-boxed security review of K3 was conducted through the Pashov Audit Group bug-hunt program. During the timeline of 3 full focused audit days, plus 1 day of fixes review, 1 issue was found across the codebase. Security Research team includes BengalCatBalu, InfiniteSec, K42, merlinboii, shaka.

About Pashov Audit Group

Pashov Audit Group is a security research collective focused on smart-contract and protocol security. This report documents the issues surfaced during a time-boxed review delivered through our competitive bug-hunt program.

METHODOLOGY

Researchers independently reviewed the in-scope code and reported the issues they found. Each submission was then triaged and validated by the judging team; duplicates were merged and severities were finalized. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

DISCLAIMER

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue was reviewed after the hunt; the outcome is recorded in the finding's Resolution section.

About the Project

The project is an asynchronous, epoch-based ERC-7540/ERC-4626 vault wrapper for assets held in a manually managed smart account (Safe), covering deposit/redemption staging, NAV settlement, user claims, operator authorization, and pausing.

It uses an upgradeable beacon-proxy architecture, with pending requests and redemption reserves held in a dedicated staging contract. Requests are grouped into epochs and priced together once the smart account closes an epoch and supplies a trusted NAV snapshot; users then claim their settled shares or assets.

PAG COMMENT

The code is generally well structured, backed by detailed architecture documentation, and supported by tests. Overall, the codebase is consistent and thoughtfully organized.

Risk Classification

- Critical** Loss or compromise of critical assets, or protocol-wide impact. Must be fixed before launch.
- High** Significant material loss of assets, or serious harm to a group of users.
- Medium** Moderate material loss, or moderate harm, often under conditional assumptions.
- Low** Minor impact, limited scope, or requires unlikely conditions to trigger.

Scope

- Contract code:
<https://github.com/K3-Capital/erc7540-wrapper/tree/f8a4e84a6ad19d08067bcc4825896cfb630d0d68/src>

REVIEW COMMIT HASH

f8a4e84a6ad19d08067bcc4825896cfb630d0d68

FIXES REVIEW COMMIT HASH

bb49d8e11b282ff48002bd123bee8ac4227cbdab

Findings

SEVERITY			COUNT
Low			1
Total			1

ID	TITLE	SEVERITY	STATUS
L-01	ERC-7540 Deposit events identify the operator instead of the controller	Low	Fixed

Low Severity

[L-01] ERC-7540 Deposit events identify the operator instead of the controller

Low Fixed

SUMMARY

The controller-aware `deposit` and `mint` overloads correctly load the requested controller and pass it to the shared `_consumeDepositClaim`, but that shared path emits the event using `_msgSender()` as the first indexed address:

```
emit Deposit(_msgSender(), receiver, assets, shares);
```

ERC-7540 requires the controller-aware `Deposit` event to use `controller` as its first indexed address and `receiver` as its second.

When the caller and controller are the same account (a direct claim), the defect is hidden because the two values are equal. When an approved operator claims on behalf of a different controller, `_msgSender()` is the operator, so the event names the operator rather than the controller that authorized and owns the claim. Token accounting and authorization remain correct.

IMPACT

During an operator-assisted claim, balances and authorization stay correct but the emitted `Deposit` event identifies the operator as the controller.

Indexers, accounting systems, and monitoring that reconcile `DepositRequest` and `Deposit` by the first indexed address can attribute the claim to the wrong account, leave the real controller's request looking still-open, or record incorrect account activity.

This is a standards-compliance and integration-integrity issue, not direct balance theft.

RECOMMENDATION

Emit the `controller` parameter as the first indexed address in the shared claim path:

```
emit Deposit(controller, receiver, assets, shares);
```

Keep the existing controller-based accounting and add an event-level regression test covering both direct-controller and approved-operator callers. If any indexer has already consumed the incorrect events, document a reconciliation or migration procedure.

RESOLUTION

K3

Acknowledged and fixed in <https://github.com/K3-Capital/erc7540-wrapper/pull/33>

fixed-PAG